哈爾濱工程大學

第 17 讲 / 全 17 讲

C 语言预处理与多文件工程详解

编译前的文字替换工，让代码从一页长成一座工程

宏定义

#define 起个名字，全文替换

文件包含

#include 把别人的声明搬进来

头文件

.h 是说明书，.c 是实现

多文件

extern 共享，static 私有

主讲教师：郎大鹏

本节目标

哈爾濱工程大學

1. 理解预处理器：它在真正的编译之前，只做文本替换
2. 会写常量宏与带参宏，并知道为什么要层层加括号
3. 分清 #include 尖括号与双引号的搜索路径差异
4. 掌握 .h 与 .c 的分工，能用 #ifndef 保护头文件
5. 用 extern 跨文件共享、用 static 把名字关在本文件
6. 读懂预处理 → 编译 → 链接，会用 #if DEBUG 调试开关

01

预处理与宏定义

#include 与 #define ：编译前的文字替换工

哈爾濱工程大学

编译前的那个 " 文字替换工 "

以 # 开头的行

哈尔滨工程大学

```
// 预处理指令：都是 # 开头
#include <stdio.h>
// 这一行被整段替换掉
#define PI 3.14159
// 后面所有 PI 都换成它
int main(void) {
    double r = 2.0;
    double s = PI * r * r;
    // 替换完：3.14159 * r * r
    printf("%f", s); // 12.566360
    return 0;
}
```

它不是 C 语句

开头的行不是代码，是给预处理器的命令，所以末尾不写分号。

干活在编译前

先扫描全文做替换，替换完才交给编译器。编译器根本没见过 PI 。

只替换，不算数

它不懂优先级、不懂类型，只是把一段字符换成另一段字符。

两种 #include ：去哪儿找

两种写法只差一对符号，但预处理器的搜索路线完全不同——一个只找系统，一个先找自己。

尖括号：找系统

```
#include <stdio.h>
```

直接到编译器自带的系统目录里翻，翻不到就报错。标准库一律用它。

双引号：先找自己

```
#include "math_utils.h"
```

先在 .c 文件所在目录找，找不到再去系统目录。自己写的头文件用它。

```
//  一个源文件开头通常长这样
#include <stdio.h>        //  系统库：尖括号
#include "math_utils.h"    //  自己的：双引号
//  效果一样：把那个文件整段搬到这一行
```

常量宏：给一串字符起个名字

格式： #define 名字 替换文本

```c
#define PI 3.14159    //  名字  PI
#define N 100         //  数组大小常用
#define GREET "hello" //  也能是文字
int main(void) {
    double r = 1.5;
    double c = 2 * PI * r;
    //  替换完：  2 * 3.14159 * r
    printf("%f %d", c, N); // 9.424770 100
    printf("%s", GREET);     // hello
    return 0;
}
```

宏名一律大写

PI 、 N 、 MAX 全大写是约定：看到大写就知道它不是变量，是会被替换掉的名字。

替换的是 " 文本 "

3.14159 在宏眼里不是 double ，只是九个待替换的字符，没有类型可言。

管到文件末尾

从 #define 那一行起，到文件结束（或遇到 #undef ）都有效。

宏 与 const ：不是一回事

两者都能 " 起名字代表一个不变的值 " ，但一个在编译前就消失，一个是真正的变量。

宏：编译前就没了

```
#define SIZE 10
```

没有类型、不占内存，编译器从头到尾没见过 SIZE 这个名字。

const ：只读的变量

```
const int LEN = 10;
```

有类型、占内存，编译器认识它，改它会报错，能参与类型检查。

C 阶段怎么选

```
int a[SIZE];
```

数组大小、编译开关、跨文件常量，用宏最省事也最通用。

```
#define SIZE 10          //  预处理完就不存在
const int LEN = 10;        //  一直活到运行时
int a[SIZE];             //  换成 int a[10]; 没问题
//  记住：宏替换发生在任何类型检查之前
```

为什么要费劲写宏

写宏的本质动机只有两个：把 " 散落各处的同一个数 " 收拢到一个地方，以及把 " 看不懂的数字 " 换成看得懂的名字。

理由 1

改一处

PI 想从 3.14 提到 3.14159 ，只改 #define 那一行，全文跟着变。

理由 2

见名知义

满屏的 3.14159 不知道是什么，写成 PI 一眼就懂是圆周率。

理由 3

统一口径

数组大小 N 写死在十几个地方，改一处漏一处就出 bug 。

理由 4

零开销

宏在编译前就替换完，运行时不占内存、不花调用开销。

一句话：凡是 " 同一个值要写很多遍 " 或 " 这个数字别人看不懂 " 的地方，就该考虑宏。

头号大坑：宏不加括号

N*N 不是 64，是 23

哈尔滨工程大学

✗ 这样写要出事

```
#define N 5 + 3
int a = N * N;
// 替换后：int a = 5 + 3 * 5 + 3;
//          = 5 + 15 + 3 = 23
printf("%d", a);   // 23
```

✓ 整体加一层括号

```
#define N (5 + 3)
int a = N * N;
// 替换后：int a = (5 + 3) * (5 + 3);
//          = 8 * 8 = 64
printf("%d", a);   // 64
```

为什么错

预处理器只管替换字符，完全不知道外面还有个乘号等着。先乘除后加减一算，就成了 23 。

铁律

宏体只要有运算，就把整个宏体用括号包起来，让它成为一个 " 铁板一块 " 的整体。

02

带参宏与头文件

能带参数的宏 · .h 与 .c 的分工 · 头文件保护

哈爾濱工程大学

带参宏：长得像函数，其实不是

呼应函数课

哈尔滨工程大学

```c
#include <stdio.h>
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int main(void) {
    int x = 3, y = 7;
    int m = MAX(x, y);
    // 替换完：((x) > (y) ? (x) : (y))
    printf("%d", m);          // 7
    double p = MAX(1.5, 2.5);
    printf("%f", p);          // 2.500000
    return 0;
}
```

只是 " 填空 "

MAX(x, y) 里 a 换成 x 、 b 换成 y ，整段塞回原处，没有调用、没有返回。

参数不求值

函数是先算出实参再传进去；宏是把表达式原样搬过去，算几次由展开式决定。

不挑类型

同一个 MAX 喂 int 、喂 double 都行——它压根不做类型检查，函数做不到这点。

带参宏：两层括号都要加

参数括号 + 整体括号

哈尔滨工程大学

✗ 漏了括号

```
#define SQR(x) x * x
int a = SQR(2 + 3);
// 替换后：int a = 2 + 3 * 2 + 3;
//          = 2 + 6 + 3 = 11
printf("%d", a);   // 11
```

✓ 两层括号

```
#define SQR(x) ((x) * (x))
int a = SQR(2 + 3);
// 替换后：int a = ((2 + 3) * (2 + 3));
//          = 5 * 5 = 25
printf("%d", a);   // 25
```

错在哪

x 只是个占位符，传进来的 2 + 3 原样粘进去，乘号先跟 3 结合，答案就跑偏了。

口诀

每个参数各自套一层括号，再把整个宏体套一层括号。宁可多写，不可少写。

副作用：i 被加了两次

MAX(i++, j) 的代价

哈尔滨工程大学

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int i = 3, j = 5;
int m = MAX(i++, j);
// 展开后长这样：
int m = ((i++) > (j) ? (i++) : (j));
// 比较时 i++ 一次，选中分支又 i++ 一次
printf("%d %d", m, i);  // 5 5
// 若是函数就只加一次，i 应该是 4
// 结论：宏参数别写 ++ 和 --
// 想自增就单独一行：i++; 再 MAX(i, j);
```

为什么两次

i++ 在展开式里出现了两回：一次在比较部分，一次在被选中的结果部分。

函数的做法

函数会先算出实参的值存进形参，i++ 只执行一次，这就是两者的根本区别。

避坑办法

宏参数只写普通的变量或常量；要改变量就另起一行写，别塞进宏里。

头文件：声明的集中地

函数课的伏笔

哈爾濱工程大學

```c
// math_utils.h —— 声明集中地
int add(int a, int b);
int max2(int a, int b);
double circleArea(double r);
#define PI 3.14159
// main.c —— 一句就全拿到
#include "math_utils.h"
int main(void) {
    printf("%d", add(3, 5));  // 8
    printf("%d", max2(3, 5)); // 5
    return 0;
}
```

它装什么

函数声明、宏定义、类型定义。就是函数课说的那句 " 先声明后使用 " 的声明。

省得手抄

没有头文件，每个 .c 都得把同一堆声明抄一遍，抄错一个字就编译不过。

改一处就够

函数签名要调整，只改 .h 里那一行，所有包含它的 .c 自动同步。

.h 是说明书，.c 是实现

买电器会拿到两样东西：说明书告诉你有什么功能，机器本身负责干活。 .h 和 .c 就是这个分工。

.h 说明书

放：函数声明、宏、类型定义。

不放：函数体、全局变量定义。

可以被任意多个 .c 用 #include 拿走。

.c 实现

放：函数体、全局变量的定义。

不放：对外声明（那属于 .h ）。

单独编译成目标文件，永远别被 include 。

两边要对得上

.h 里声明的签名和 .c 里实现的签名必须一模一样，否则链接时找不到人。

```
// math_utils.h：只说有什么
int add(int a, int b);
// math_utils.c：负责怎么做
int add(int a, int b) { return a + b; }
```

#ifndef ：头文件的防盗门

防重复包含

哈尔滨工程大学

```
// math_utils.h 的标准写法
#ifndef MATH_UTILS_H    // 没定义过?
#define MATH_UTILS_H    // 那就定义一下
int add(int a, int b);
int max2(int a, int b);
#define PI 3.14159
#endif                  // 到此为止
// 第二次被 include 时，宏已定义，
// #ifndef 为假，整段跳过，不会重复
// 宏名习惯：文件名全大写，点变下划线
// student.h -> STUDENT_H
```

不保护会怎样

同一个 .c 里绕来绕去包含了两次，同一份声明出现两遍，编译器直接报重复定义。

三板斧

#ifndef 开头问一句，#define 做个记号，#endif 收尾。三行固定搭配。

养成习惯

以后每新建一个 .h ，第一件事就是把这三行写上去，再写正式内容。

03

多文件工程

拆文件 · extern 与 static · 编译链接流程 · 条件编译

哈爾濱工程大学

为什么非要把文件拆开

前面 13 套课件写的都是单个 .c 文件。真做课设时一个文件塞到几百行，麻烦会一个接一个冒出来。

痛点 1

找不到

两千行的文件里找一个函数，滚动条拖半天，上下对照更是折磨。

痛点 2

全重编

只改了一行，整个文件都要重新编译一遍，文件越大等得越久。

痛点 3

抢一处

两个人同时改同一个文件，互相覆盖；拆开后各改各的，互不打扰。

痛点 4

搬不走

这次写的工具函数想用到下个程序里，只能整段复制粘贴，改 bug 要改两处。

拆分的依据：按 " 干什么 " 分。同一类功能的函数放一个 .c ，配一个同名 .h 对外说明。

三件套：.h + .c + main.c

最小可运行工程

哈尔滨工程大学

math_utils.h

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
int max2(int a, int b);
#endif
```

只放声明，谁都可以 include 它

math_utils.c

```
#include "math_utils.h"
int add(int a, int b) {
    return a + b;
}
int max2(int a, int b) {
    return a > b ? a : b;
}
```

函数体在这里，自己也要 include 同名 .h

main.c

```
#include <stdio.h>
#include "math_utils.h"
int main(void) {
    printf("%d", add(3, 5));
    printf("%d", max2(3, 5));
    return 0;
}
```

编译： gcc main.c math_utils.c -o app

extern ：借用别处的全局变量

声明不等于定义

```c
// config.c —— 真正的定义，分配内存
int g_count = 0;
// main.c —— 只想用，不再造一个
extern int g_count;
int main(void) {
    g_count = g_count + 1;
    printf("%d", g_count);  // 1
    return 0;
}
// 编译：gcc main.c config.c -o app
// 链接时 linker 把两边的 g_count 对上
```

借，不是造

extern 只是告诉编译器 " 这个名字在别处已经定义了 " ，自己不分配内存。

定义只有一次

不带 extern 的 int g_count = 0; 全工程只能出现一次，多了链接报错。

常见写法

把 extern 声明写进 .h ，谁要用谁 include ，省得每个 .c 都写一遍。

static ：把名字关在本文件

```
// tool.c
static int secret = 10;     //  只本文件可见
static void helper(void) { //  只本文件能调
    printf("%d", secret);
}
void openIt(void) {         //  不给 static：对外
    helper();                  //  内部随便用
}
// main.c 里就算写 extern int secret; 也拿不到，
//  链接时报错：找不到这个符号
//  但 main.c 可以正常调用 openIt()
```

static 变量

全局变量加了 static ，作用域就被锁死在这个 .c 里，别的文件看不见也借不走。

static 函数

内部辅助函数一律加 static ，等于对外宣布 " 这是我家私事，别调用 " 。

最大的好处

两个文件都叫 helper 也不打架——各自是各自的，工程越大越省心。

从 .c 到可执行程序：三步走

本课核心图

哈尔滨工程大学

点一下 " 编译 " 按钮，背后其实经历了预处理、编译、链接三个阶段，宏只在第一阶段存在。

第 1 步

预处理

展开 #include 、替换 #define 、处理条件编译，生成一份纯 C 文本。

第 2 步

编译

检查语法、翻译成机器指令，每个 .c 各自生成一个 .o 目标文件。

第 3 步

链接

把所有 .o 和库函数拼在一起，对上跨文件的名字，生成可执行程序。

产物

app.exe

链接阶段才查 " 你说的 add 到底在哪个文件里 " ，找不到就报错。

```
gcc -E main.c -o main.i          //  只看预处理结果
gcc -c main.c -o main.o          //  只编译，先不链接
gcc main.o math_utils.o -o app //  把两个 .o 链接起来
gcc main.c math_utils.c -o app //  一步到位
```

条件编译：一个开关控整段代码

#if DEBUG

```c
#include <stdio.h>
#define DEBUG 1        // 开关：1 开 0 关
int main(void) {
    int x = 10;
#if DEBUG
    printf("x = %d", x);  // 调试时才留
#endif
    printf("done");
    return 0;
}
// DEBUG 改成 0，那句 printf 直接消失
```

#if 怎么判断

后面跟常量表达式，非 0 就留下中间那段，为 0 就在预处理时整段删掉。

和 if 不一样

if 是运行时判断，两条路的代码都在程序里；#if 是编译前决定，删掉的压根不存在。

常用在哪

调试日志、临时测试代码、不同平台走不同分支，都是条件编译的地盘。

例 1 · 用 PI 宏算圆面积

哈爾濱工程大學

例 1 读入半径 r ，用 PI 宏计算圆面积，保留两位小数输出

```c
#include <stdio.h>
#define PI 3.14159
int main(void) {
    double r, area;
    scanf("%lf", &r);     // 输入 3
    area = PI * r * r;
    // 替换完: area = 3.14159 * r * r;
    printf("%.2f", area);  // 28.27
    return 0;
}
```

运行结果

输入： 3
输出： 28.27
输入： 10 -> 314.16

要点

PI 写在文件开头，全文随时可用，不必反复写 3.14159 。

想提高精度，只改 #define 那一行，面积公式不用动。

宏名大写，一眼能认出它不是变量。

例 2 · MAX / MIN / ABS 三个宏

例 2 写出求较大值、较小值、绝对值的三个带参宏，并实际调用

```c
#include <stdio.h>
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#define ABS(x) ((x) >= 0 ? (x) : -(x))
int main(void) {
    int x = -7, y = 4;
    printf("%d", MAX(x, y));  // 4
    printf("%d", MIN(x, y));  // -7
    printf("%d", ABS(x));     // 7
    return 0;
}
```

运行结果

```
MAX(-7, 4) = 4
MIN(-7, 4) = -7
ABS(-7)    = 7
```

要点

每个参数都套括号，整个宏体再套一层，这是标准写法。

同一个宏喂 int 或 double 都能用，函数做不到这点。

参数里别写 i++ 这种，会被展开两次。

例 3 · 把 23 修回 64

括号陷阱修复

哈尔滨工程大学

例 3 #define N 5 + 3 之后 N*N 算成了 23 ，请修复并说明原因

```c
#include <stdio.h>
#define N_BAD 5 + 3          //  错：没括号
#define N_OK (5 + 3)          //  对：整体括号
int main(void) {
    printf("%d", N_BAD * N_BAD);
    //  展开：5 + 3 * 5 + 3   ->  23
    printf("%d", N_OK * N_OK);
    //  展开：(5 + 3) * (5 + 3)   ->  64
    return 0;
}
```

运行结果

```
N_BAD * N_BAD = 23
N_OK  * N_OK  = 64
```

修复要点

宏体含运算，就把整个宏体用括号包成一个整体。

带参宏还要给每个参数各加一层括号。

记不住就记一句：宁可多写括号，不要少写。

例 4 · 三文件拆分

把 sumTo(n) 拆出 main.c

哈爾濱工程大學

math_utils.h

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int sumTo(int n);
#endif
```

对外只有一句声明，谁要用谁 include

math_utils.c

```
#include "math_utils.h"
int sumTo(int n) {
    int i, s = 0;
    for (i = 1; i <= n; i++)
        s = s + i;
    }
    return s;
}
```

签名与 .h 里那句必须完全一致

main.c

```
#include <stdio.h>
#include "math_utils.h"
int main(void) {
    printf("%d", sumTo(100));
    printf("%d", sumTo(50));
    return 0;
}
```

输出 5050 1275 ； gcc main.c math_utils.c -o app

例 5 · DEBUG 开关控制日志

条件编译实战

哈爾濱工程大學

```c
#include <stdio.h>
#define DEBUG 1        // 1 开 0 关
#if DEBUG
#define LOG(v) printf("s=%d", v)
#else
#define LOG(v)             // 什么都不做
#endif
int main(void) {
    int s = 0, i;
    for (i = 1; i <= 3; i++) {
        s = s + i;
        LOG(s);            // 看每一步
    }
}
```

题目

累加 1 到 3，用 DEBUG 开关决定要不要打印每一步的中间结果

运行结果

```
DEBUG 1: s=1s=3s=6
DEBUG 0: 什么也不打印
```

要点

DEBUG 为 0 时 LOG 变成空宏，那行代码等于被删掉。

调试完把 1 改成 0 就收工，不必手动删 printf。

正式发布的版本里一点调试代码都不留。

常见错误盘点

这六条最容易丢分

哈尔滨工程大学

- ✗ 宏体忘加括号：#define N 5 + 3 ，之后 N*N 算成了 23 而不是 64
- ✗ 宏定义末尾画蛇添足加分号：#define PI 3.14159; 分号会被一起替换进去
- ✗ 带参宏的参数忘加括号：#define SQR(x) x * x ， SQR(2+3) 得到 11
- ✗ .h 里的声明和 .c 里的实现签名对不上，编译能过、链接报错
- ✗ 写了 #ifndef 却漏掉 #endif ，或者漏掉中间那句 #define 标记
- ✗ extern 和定义搞混：在 .h 里直接写 int g = 0; 每个包含的 .c 都造一份

14 套课件，一路走到这里

printf 与类型 → scanf 输入 → if 家族 → for 循环 → while 循环 → 函数 → 递归 → 数组 → 指针 → 排序查找 → 字符串 → 结构体 → 文件 → 预处理与多文件工程

本课三件事

宏是纯文本替换；头文件是说明书；多文件靠 extern 和 static 划界限。

记住一句话

凡是宏，先想 " 替换之后长什么样 " ，括号问题自然就避开了。

接下来

数据结构、操作系统、算法与竞赛， C 语言只是起点。